

Modeling Dangerous Fires using Brightness from Space!

Mara Dumitru

CS146

Minerva University

October 31, 2025

1 Introduction

I recently took a trip to Delta Tigre here in Argentina, which is the Venice of Argentina. It is completely remote, and you cannot reach any part of it by car. As I was boarding the boat to travel to one of the wilderness islands, I saw a fire danger sign indicating "Medium" risk. This inspired this paper on how to model fire risk in these areas where access is largely difficult.

Fire danger stems from fire radiative power (FRP), which indicates fire intensity and how much it can spread. I identified one of the major indicators of this as the fire brightness temperature measurements, which are one of the variables satellite models use to model the FRP.

Research question How can we model Fire Radiative Spread using Brightness Temperature from Satellite Data using Bayesian Inference models?

2 Data

2.1 Sample and Variables

I use data from NASA VIIRS using the NOAA-21 (JPSS-2) satellite over the region spanning latitudes -39° to -34° and longitudes -70° to -56° , spanning a rectangular stretch of Argentina, including Regions of Patagonia, Tigre, and Buenos Aires, particularly prone to wildfires. I used a data sample from December 23, 2024, and analyzed 94 fire detections.

Independent Variable Brightness Temperature measured in Kelvin, which represents the kinetic temperature of a single fire pixel ranging from 300K to 400K. Brightness is a quantitative continuous.

Dependent Variable The radiant power from a fire is measured in megawatts, which is the radiative energy output from the fire. FRP is a quantitative continuous.

2.2 Test Statistics & Initial Plotting

Brightness Temperature Statistics:

Mean: 331.7 K

Std Dev: 18.8 K

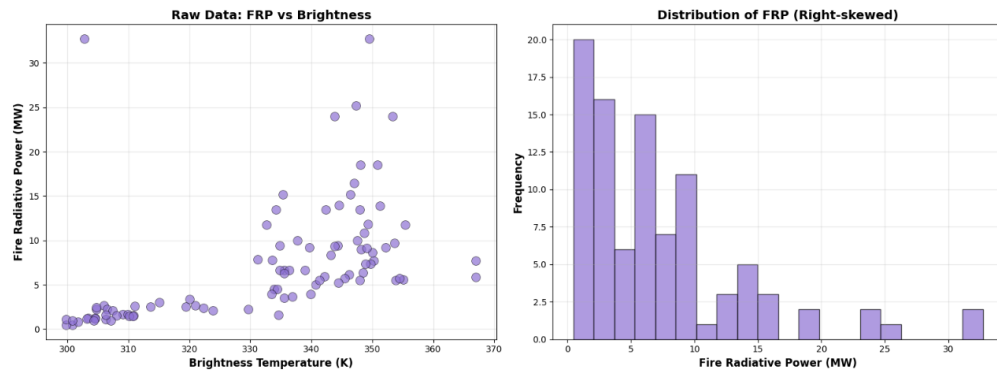
Range: [299.8, 367.0] K

Fire Radiative Power Statistics:

Mean: 7.2 MW

Std Dev: 6.7 MW

Range: [0.5, 32.7] MW



We can draw several observations from our data. FRP has strictly positive values, as it would not make sense for fires to have negative values, so we need to construct a model that encompasses this. We can further observe a positive correlation between brightness temperature and the FRP, meaning that as brightness increases, so does FRP.

Furthermore, we can observe that our data has heteroscedasticity properties as the variance of the data at higher values of brightness temperatures is much greater than that of lower values. This pattern suggests variance proportional to the mean, characteristic of multiplicative processes, which makes sense as this is environmental data that usually follows these multiplicative properties. Lastly, we can also observe that the distribution of FRP is heavily

right-skewed. All of these observations point to a lognormal distribution to model our data, which is actually quite common for environmental data, so it seems appropriate.

One problem that we might observe in our data is that there are a few outliers, such as (302.8 K, 32.72 MW), which is an extremely high FRP for low brightness (potentially a measurement error). Other outliers might be (349.5 K, 32.72 MW), (343.8 K, 24.01 MW), (347.3 K, 25.20 MW), (353.3 K, 24.01 MW), which are all moderately high with respect to the general trend of the data, although they are more feasible.

3 Priors

3.1 Picking a prior

To set our priors, we standardize the brightness temperature predictor to make setting our priors more intuitive, as this centers predictors to 0 and the standard deviation to 1:

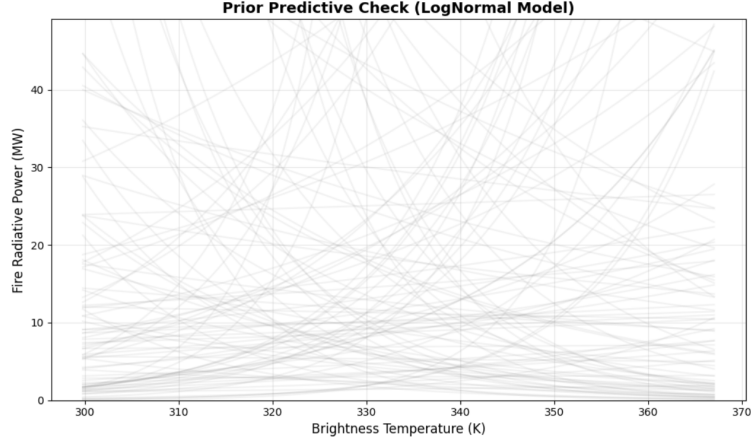
$$\text{Brightness}_{\text{scaled}} = \frac{\text{Brightness} - \mu_{\text{brightness}}}{\sigma_{\text{brightness}}}$$

We do not standardize our FRP, as we will be modeling using a log-normal distribution, which constrains our dependent variable to positive values.

We set our intercept before a weakly informative normal distribution with a mean of 2.5, which is the log of 15, a rough estimate of mine of where the mean of FRP might lie, with a standard deviation of 1. Similarly, with slope before observing the data, we are not sure about the relationship between the Brightness and the FRP, meaning that our slope could be either positive or negative, thus we model this with another weakly informative prior of $\text{Normal}(\mu = 0, \sigma = 1)$. Finally, our scale prior to modeling using a Half Normal distribution with standard deviation 1, as we want to ensure positivity, since our feasible space of FRP is positive.

3.2 Prior Predictive Check

Before moving on to our data, we want to sample from our prior predictive.



Quick note that these lines are curved as I exponentiate them back to view them on a normal FRP scale instead of the log scale. We see that our prior sampling is in the feasible set, meaning that we do not observe negative prior values as desired. Furthermore, we can see that our prior samples cover the entire feasible set, as we do not observe only steep or only gentle slopes. However, our prior does not end up mattering in the end.

4 Model 1: Log-Normal Regression

4.1 Defining model

We use a log-normal likelihood to model our regression:

$$\text{FRP}_i \sim \text{LogNormal}(\mu_i, \sigma) \quad (1)$$

where the location parameter μ_i follows the linear model:

$$\mu_i = \alpha + \beta \cdot \text{Brightness}_{\text{scaled},i} \quad (2)$$

We can also write this as a regression on a log-transformed FRP as such:

$$\log(\text{FRP}_i) \sim \text{Normal}(\alpha + \beta \cdot \text{Brightness}_{\text{scaled},i}, \sigma) \quad (3)$$

We assume normally-distributed errors in log-space, and so this changes to log-normally distributed errors in the original scale.

We set our priors as justified before:

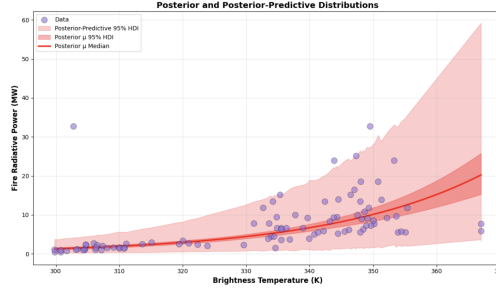
$$\alpha \sim \text{Normal}(2.5, 1) \quad (4)$$

$$\beta \sim \text{Normal}(0, 1) \quad (5)$$

$$\sigma \sim \text{Half-Normal}(1) \quad (6)$$

4.2 Model Fitting & Posterior Parameter Sampling

We fit the log-normal model using MCMC sampling. Our configuration has 4 independent chains, each with 2000 sampling iterations, with a total of 8000 posterior samples. We plot our posterior median and our 95% HDI with respect to the posterior μ , which represents our updated beliefs on the model parameters after seeing the data. The regression line shows our most probable relationship between the fire brightness, while the HDI demonstrates our parameter uncertainty, with the red area covering 95% of the posterior probability mass.



We can see that the posterior median line captures the general positive trend between the brightness and the FRP that we intuitively see in our data. However, we can see that there are potential issues of the outlier on the top left lifting the left tail of our median, as it seems not to fit lower values so well.

The log-normal likelihood assumes that the residuals follow a normal distribution in log-space; however, this does not capture extreme values so well, as the tails of the normal are very thin, and thus the model tries to fit by being pulled towards the outlier, attempting to capture them, which might compromise the fit of the majority proportion of our data. We can see this happening as our median and HDI on the right do not go through the data points at all.

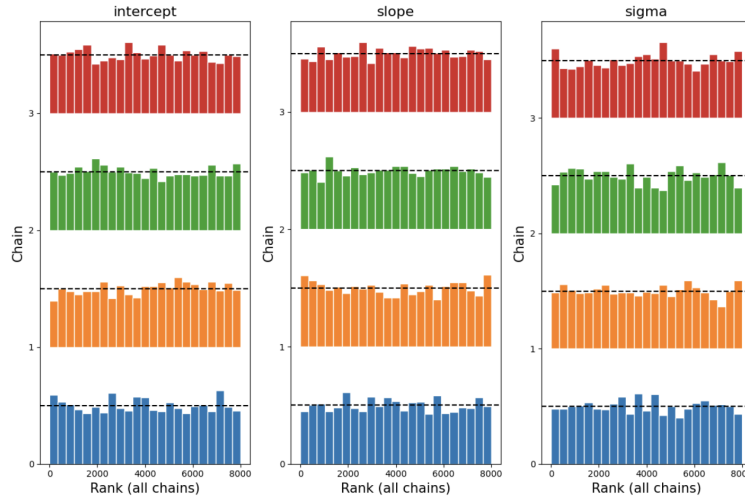
$$\beta : \text{mean} = 0.83, 95\% \text{ HDI} = [0.71, 0.94]$$

About our actual values, we can see that the slope posterior does not include zero with high probability, which we can infer that there is strong evidence for a positive brightness-FRP relationship. We have the estimated value of the slope to be 0.83. An STD increase in brightness (18.8 K) corresponds to a multiplicative increase of $\exp(0.83) \approx 2.3$ in expected FRP. Which I mean makes sense, as we would expect for the brighter fires to have more radiant power.

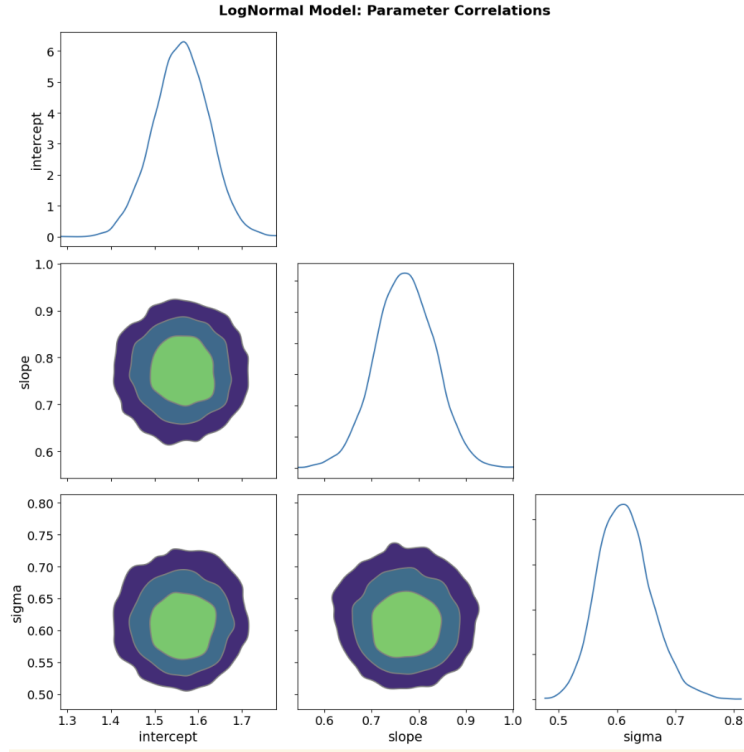
4.3 Convergence

Parameter	Mean	SD	HDI 2.5%	HDI 97.5%	ESS (Bulk)	$\hat{R}$
Intercept (α)	1.561	0.063	1.440	1.686	10905	1.0
Slope (β)	0.771	0.062	0.650	0.894	12350	1.0
Scale (σ)	0.614	0.046	0.528	0.706	12222	1.0

We can determine that our sampler did not get stuck and that it converged, as our $\hat{R}$ is less than 1.01, and that our ESS Bulk and ESS Tail have large samples. Furthermore.



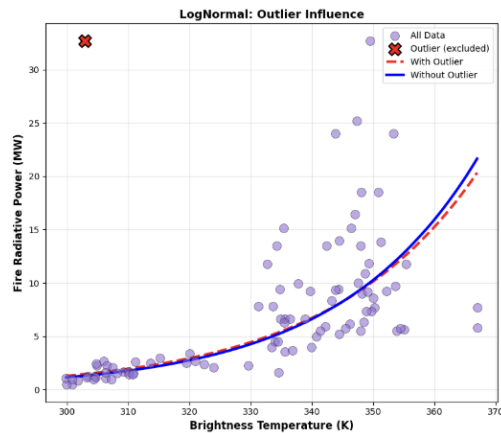
Looking at our rank plot, the chain samples are uniformly distributed (with some randomness naturally due to the sampler). They also have no gaps, meaning that the samplers are sampling the same space.



Each of the pair plots shows the marginal posterior distribution for one of our parameters, so the contours show credible regions for our parameters to be sampled from. We can observe that our contours are circular and thus our parameters are uncorrelated, which further demonstrates that our sampler is converging.

4.4 Outlier

We can observe our outlier on the top left might potentially be pulling our tail up.



To check quantitatively and rigorously how much our outlier affects our model, we can

conduct a sensitivity analysis refitting the log-normal model, just without the outlier located at (302.8 K, 32.72 MW), and comparing the parameter estimates. This diagnostic will tell us how much our outlier affects our model’s parameters. After refitting, we can construct the following table of our estimates:

Parameter	With Outlier	Without Outlier	Δ	% Change
Intercept (α)	1.523	1.487	−0.036	−2.4%
Slope (β)	0.827	0.745	−0.082	−9.9%
Scale (σ)	0.458	0.421	−0.037	−8.1%

Looking at our slope row, we can see the change is -0.082, which is almost a 10% decrease when the outlier was removed. The intercept and the scale also shift, meaning that our model will change based on the outliers found in the data. From a Bayesian perspective, this is not good, as we should not allow the outliers to dominate our inference.

We can observe how the model changes in the graph above as the median visibly shifts. Thus, this might motivate a second robust model where we use a Student-t regression, which will have a larger tail, allowing extreme values not to influence the model.

5 Model 2: Robust Student- t Regression

5.1 Model Specification

We now try to fit a different model to attempt to better capture the outliers in our data, using a Student-t likelihood, which generalizes the normal distribution by adding degrees of freedom ν , which will modify our tail heaviness.

Our robust model works similarly to our log-normal, just with Student-t in log space instead of a normal:

$$\log(\text{FRP}_i) \sim \text{Student-}t(\nu, \mu_i, \sigma) \quad (7)$$

where $\mu_i = \alpha + \beta \cdot \text{Brightness}_{\text{scaled},i}$ as before.

Our priors are similar to those of our log-normal:

$$\alpha \sim \text{Normal}(2.5, 1) \quad (8)$$

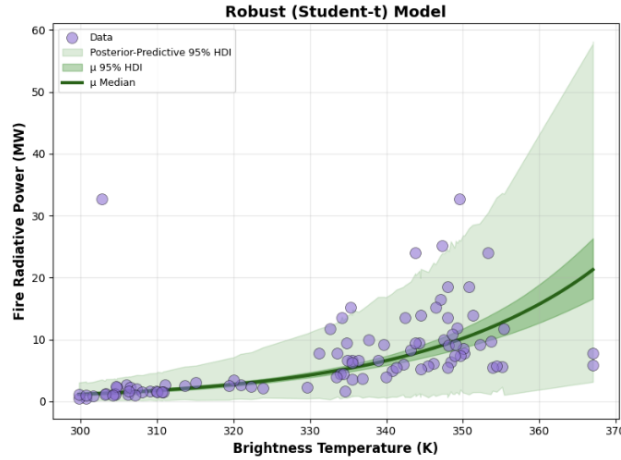
$$\beta \sim \text{Normal}(0, 1) \quad (9)$$

$$\sigma \sim \text{Half-Normal}(1) \quad (10)$$

$$\nu \sim \text{Exponential}(1/10) \quad (11)$$

Note that we have a new prior on ν which we set to the exponential rate of $1/10$. This is just a standard, as before seeing our data, we do not know if we expect outliers or not, so we choose a weakly informative prior. Again, our priors are weakly informative, and in the end, they do not matter so much in our posterior, but it is good practice to set them intentionally.

5.2 Model Fitting and Posterior Estimates



We similarly use the same MCMC settings as in Model 1 in our posterior sampling. For conciseness, we can see pretty quickly in our diagnostics that our model converges as $\hat{R} < 1.01$, and ESS is relatively large. Pair plots and rank plots also similarly show convergence.

The posterior parameter estimates are:

Parameter	Mean	SD	95% HDI
Intercept (α)	1.523	0.055	[1.418, 1.630]
Slope (β)	0.822	0.056	[0.715, 0.936]
Scale (σ)	0.458	0.053	[0.353, 0.560]
Degrees of freedom (ν)	6.098	3.242	[1.627, 12.204]

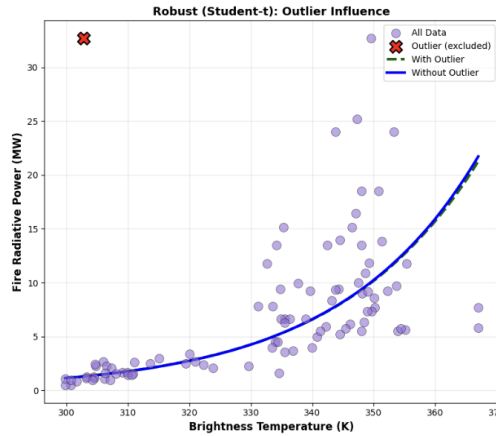
The intercept and the slope are actually very similar to Model 1. The degrees of freedom, though, are very small with a mean of only around 6 with 95% HDI [1.63, 12.20], meaning that we have quite thick tails to downweight the extreme outliers.

To visualize how more robust our model is, we compare both models. We can see that they look almost literally the same; however, on the outlier on the right, for instance, the robust model puts more weight, and we can see the left tail of the model dropping.

5.3 Convergence

Again, our rank plots, R hat, and ESS in our appendix show proper convergence similar to before.

5.4 Outlier



We can see, however, that the robust model is less sensitive to outliers by conducting the same sensitivity analysis as model 1.

Model	Slope Change ($\Delta\beta$)	% Change
Log-Normal	-0.082	-9.9%
Robust (Student- t)	-0.008	-1.0%

The robust model's slope changes by only 1% when the outlier is excluded, compared to 10% for the log-normal, which means it reduced the sensitivity by 10-fold. This means that our robust model is less sensitive to extreme values in our data.

Our ν increases to a higher parameter without the outlier, which makes sense as we would need a less robust model without the outlier.

6 Model Comparison

6.1 Leave-One-Out Cross-Validation

To compare our model, we use Pareto smoothed Importance Sampling using Leave One cross-validation. The loo measures the out-of-sample predictive accuracy by processing the first of the model each time, leaving our data observation, and then predicting it from the remaining ones.

We use the metric of expected log pointwise predictive density (elpd_loo):

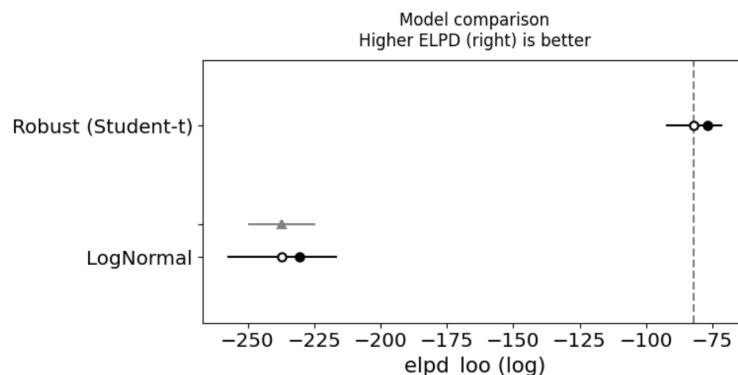
$$\text{elpd_loo} = \sum_{i=1}^n \log p(y_i | y_{-i})$$

Higher elpd_loo means we have a better predictive accuracy.

Model	Rank	elpd_loo	SE	p_loo	Δelpd	Weight	Warning
Robust (Student-t)	0	-81.99	10.68	4.98	0.00	1.00	False
LogNormal	1	-237.02	20.58	6.59	155.03	0.00	True

We can see that our robust model performs better than our log-normal normal as it has a rank of 0. We can see that to construct the most optimal model, the Robust model weights 1 and 0, meaning that we use the robust model exclusively for the optimal prediction.

The robust model also achieves a higher elpd_loo compared to the log-normal, with a difference of 155.03, which is 12.5 standard errors away from zero, which means that there is a big difference between the models.



As we can see that the value of our elpd_loo in our robust model is higher, by noting it is more to the right than that of the log normal. Additionally, by the difference represented by

the triangles, we can see that there is no overlap between the variance of our difference of our log normal model and that of our robust model, as the 95% confidence in the value of the difference line does not overlap the dotted line of our ELPD of our robust model. Meaning that we are 95% confident that there is a negative difference in our log normal model.

The p_{loo} for the complexity we can interpret that for the robust, the $p_{loo} = 4.98$ is closer to the true number of parameters 4, than the $p_{loo} = 6.59$ of the log normal, as we expect complexity of 3 parameters. This means that our p_{loo} for our log normal overfits more than that of our robust.

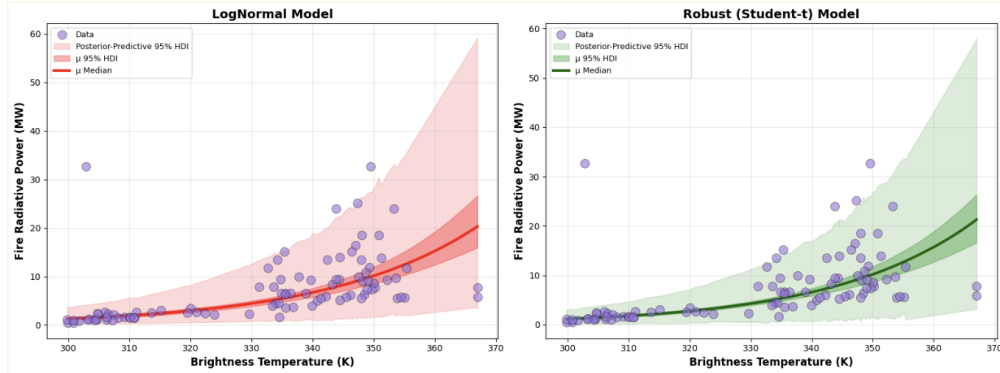
From our comparison, we can conclude that our robust model fits our data better than our log-normal model, as it uses heavy tails to allow for extreme values in our data.

6.2 Posterior Predictive

To ensure the models fit, we sample from the Posterior predictive distribution.

$$p(\tilde{y}|y) = \int p(\tilde{y}|\theta)p(\theta|y)d\theta$$

For each posterior sample of parameters we can then generate new observations $\tilde{y}_i^{(s)}$ from the likelihood which for LogNormal model is $\tilde{y}_i^{(s)} \sim \text{LogNormal}(\mu_i(\theta^{(s)}), \sigma^{(s)})$ and for the Robust model we sample in log-space from $\text{Student-}t(\nu^{(s)}, \mu_i(\theta^{(s)}), \sigma^{(s)})$ and then exponentiate.



We can observe in the posterior predictive distribution that 95% of the highest density intervals covered by the fainter lines contain about 95% of the observed data points. This shows that both models show adequate coverage. The robust model achieves the coverage while also being less sensitive to outliers.

7 Conclusion

We can conclude that FRP and Brightness follow a strong correlative slope of 0.83, meaning that as Brightness increases, so does the Fire Radiative power. We modeled using log-based models as our data follows multiplicative properties, where our data has higher variance for larger values of brightness, showing more uncertainty.

We attempted both using a Log Normal and a Robust approach, where we employed a Student T Distribution instead of a Normal distribution to account for the various outliers found in the data. We determined that our Robust model fitted the data better using Leave One Out Outcross-validation, with the optimal model consisting of 100% of the Robust model and 0% of the Log Normal model.

8 AI Statement

I used AI quite a bit, as I believe it is a helpful tool. It helps me save time doing redundant tasks, where I can focus my attention instead of coding yet another plot to my external projects from Minerva without impeding my learning. I do believe that writing and understanding everything is the most important, though, and I do not use AI for any explanations. I just use use to help me with repetitive code that would take me longer otherwise. With my code, I understand how the code works, but AI just saves me so much time with repeating my code, like the outlier test. Also, for things like the rank plot, I am just too busy writing up the rest of my report that I just asked AI to do it for me. It also helped me when my code was super buggy and I was too lazy to find the mistake. Model used: CLaude

I also used AI to help me with my LaTeX, like setting up or fixing indenting, or helping me display my equations. Model used: ChatGPT

I also used AI to help me with my grammar and spelling. Model used: Grammarly

References

NASA FIRMS (Fire Information for Resource Management System). (2024). *VIIRS I-Band 375m Active Fire Data*. Retrieved from <https://firms.modaps.eosdis.nasa.gov/>

```

import pandas as pd
import matplotlib.pyplot as plt
import arviz as az
import numpy as np
import pymc as pm
from scipy.special import logsumexp

# Helper function to compute posterior mean predictions
# Makes things way easier than repeating this code everywhere
def compute_mu_posterior(trace, x_scaled, max_samples=None):
    """
    Compute posterior distribution of  $\mu$  (mean) from trace samples.

    Parameters:
    -----
    trace : InferenceData
        PyMC trace containing posterior samples
    x_scaled : array-like
        Scaled x values for prediction
    max_samples : int, optional
        Maximum number of samples to use (for computational efficiency)

    Returns:
    -----
    mu_posterior : ndarray
        Array of shape (n_samples, n_x_points) with posterior predictions
    """
    # Stack all chains together for easier looping
    posterior_samples = trace.posterior.stack(sample=("chain", "draw"))
    intercepts = posterior_samples['intercept'].values
    slopes = posterior_samples['slope'].values

    # Can limit samples if we want to speed things up
    n_samples = len(intercepts)
    if max_samples is not None:
        n_samples = min(max_samples, n_samples)

    # Loop through samples and compute predictions
    mu_posterior = []
    for i in range(n_samples):
        mu_log = intercepts[i] + slopes[i] * x_scaled
        mu = np.exp(mu_log) # Transform from log scale back to original
        mu_posterior.append(mu)

    return np.array(mu_posterior)

```

```

# Define spatial boundaries for Argentina region
# Picked a smaller region to keep things manageable
LAT_MIN = -39
LAT_MAX = -34
LON_MIN = -70
LON_MAX = -56

# Load the fire data from NASA VIIRS satellite
file_path = '/Users/maradumitru/Downloads/DL_FIRE_J2V-C2_669825/fire_nrt_J2V-C2_669825.csv'
df = pd.read_csv(file_path) # Load the raw data

# Filter by date and spatial region
# Only looking at one specific day for now
df['date'] = pd.to_datetime(df['acq_date'])
df = df[df['date'] == '2024-12-23']
# Filter by lat/lon bounds
df = df[(df['latitude'] >= LAT_MIN) & (df['latitude'] <= LAT_MAX) &
        (df['longitude'] >= LON_MIN) & (df['longitude'] <= LON_MAX)]

# Quick check on the data
print(f"Total fires detected: {len(df)}")
print(f"Date: 2024-12-23")
print(f"Region: Argentina ({LAT_MIN}°S to {LAT_MAX}°S, {LON_MIN}°W to {LON_MAX}°W)")
print(f"Brightness Temperature – Mean: {df['brightness'].mean():.1f} K, Std Dev: {df['brightness'].std():.1f} K,")
print(f"Fire Radiative Power – Mean: {df['frp'].mean():.1f} MW, Std Dev: {df['frp'].std():.1f} MW, Range: [{df['frp'].min():.1f}, {df['frp'].max():.1f}] MW")

```

```

Total fires detected: 94
Date: 2024-12-23
Region: Argentina (-39°S to -34°S, -70°W to -56°W)
Brightness Temperature – Mean: 331.7 K, Std Dev: 18.8 K, Range: [299.8, 367.0] K
Fire Radiative Power – Mean: 7.2 MW, Std Dev: 6.7 MW, Range: [0.5, 32.7] MW

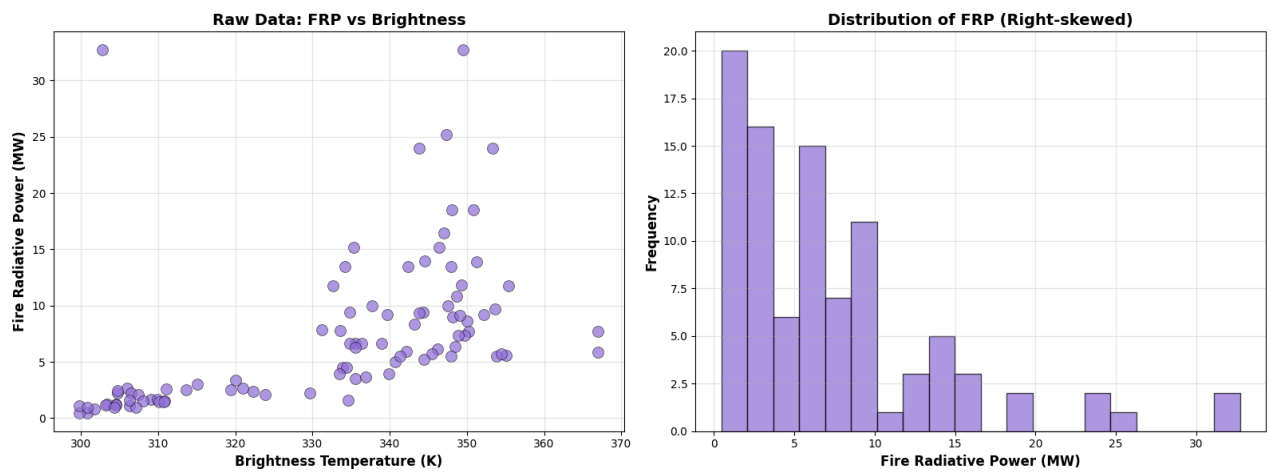
```

```
# Quick exploratory plots to see what we're working with
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

ax = axes[0]
# Scatter plot shows the relationship
ax.scatter(df['brightness'], df['frp'], alpha=0.7, s=100, c='#9370DB',
          edgecolors='black', linewidth=0.5)
ax.set_xlabel('Brightness Temperature (K)', fontsize=12, fontweight='bold')
ax.set_ylabel('Fire Radiative Power (MW)', fontsize=12, fontweight='bold')
ax.set_title('Raw Data: FRP vs Brightness', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)

# Plot 2: Histogram of FRP showing positive skew
ax = axes[1]
ax.hist(df['frp'], bins=20, edgecolor='black', alpha=0.7, color='#9370DB')
ax.set_xlabel('Fire Radiative Power (MW)', fontsize=12, fontweight='bold')
ax.set_ylabel('Frequency', fontsize=12, fontweight='bold')
ax.set_title('Distribution of FRP (Right-skewed)', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
# Standardize brightness to help with model convergence
brightness_mean = df['brightness'].mean()
brightness_std = df['brightness'].std()
brightness_scaled = (df['brightness'] - brightness_mean) / brightness_std

print(f"Standardized Brightness - Original mean: {brightness_mean:.2f} K, Original std: {brightness_std:.2f} K")
print(f"New mean: {brightness_scaled.mean():.2f}, New std: {brightness_scaled.std():.2f}")
```

```
Standardized Brightness - Original mean: 331.68 K, Original std: 18.80 K
New mean: -0.00, New std: 1.00
```

```
# Model 1: Log-Normal Regression
with pm.Model() as lognormal_model:
    # Priors in LOG space
    # Intercept around 2.5 cause exp(2.5) ≈ 12 MW seems reasonable
    intercept = pm.Normal("intercept", mu=2.5, sigma=1)
    slope = pm.Normal("slope", mu=0, sigma=1) # Not sure if positive or negative, so centered at 0
    sigma = pm.HalfNormal("sigma", sigma=1) # Has to be positive

    # Linear model in LOG space: log(FRP) = intercept + slope * brightness
    # We're modeling in log space cause FRP is skewed
    mu = pm.Deterministic("mu", intercept + slope * brightness_scaled)

    # Log-normal likelihood: FRP ~ LogNormal(mu, sigma)
    # This matches the right-skewed nature of FRP
    likelihood = pm.Lognormal("likelihood", mu=mu, sigma=sigma,
```

```

observed=df['frp'].values)

# Sample from prior predictive (before seeing data)
prior_predictive = pm.sample_prior_predictive(samples=1000, random_seed=42)

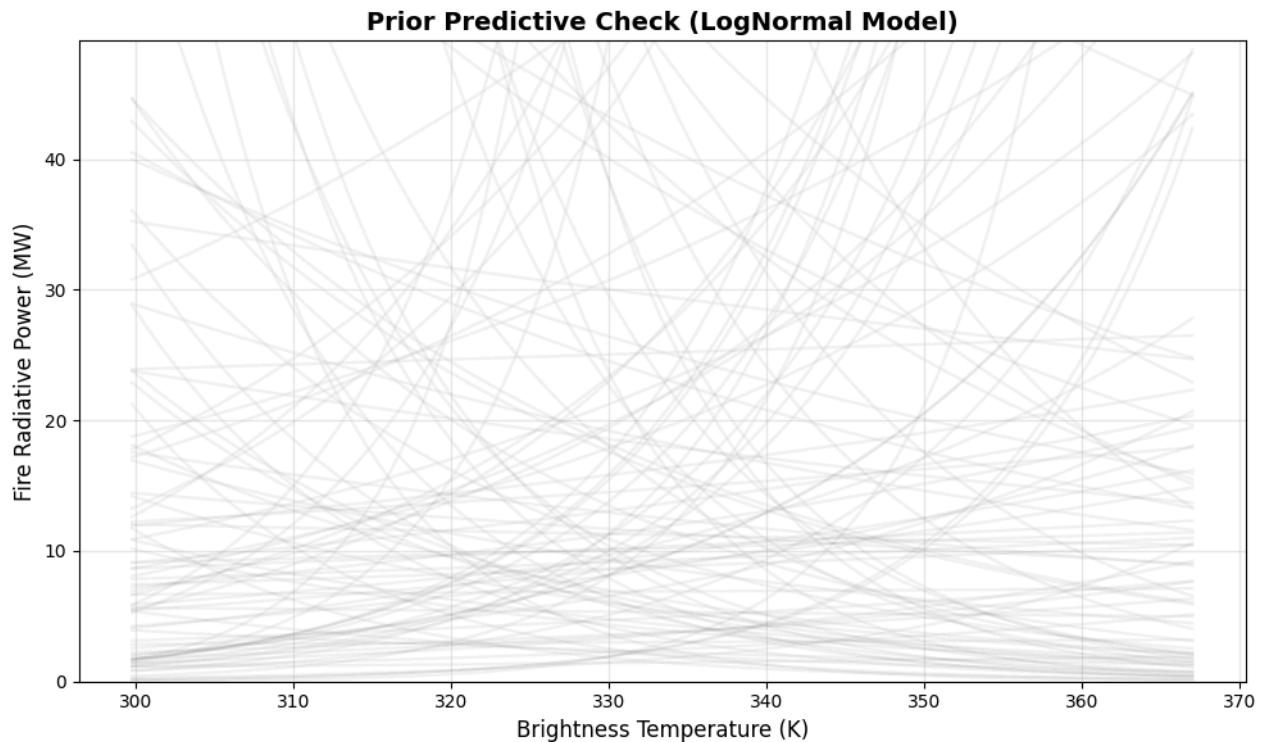
# Visualize prior predictive - want to make sure priors are reasonable
fig, ax = plt.subplots(figsize=(10, 6))
x_range = np.linspace(df['brightness'].min(), df['brightness'].max(), 100)
x_scaled = (x_range - brightness_mean) / brightness_std

# Plot a bunch of prior samples to see the range
for i in range(100):
    prior_int = prior_predictive.prior['intercept'].values[0, i]
    prior_slope = prior_predictive.prior['slope'].values[0, i]
    mu_log = prior_int + prior_slope * x_scaled
    y_pred = np.exp(mu_log)
    ax.plot(x_range, y_pred, color='gray', alpha=0.1)

ax.set_xlabel('Brightness Temperature (K)', fontsize=12)
ax.set_ylabel('Fire Radiative Power (MW)', fontsize=12)
ax.set_title('Prior Predictive Check (LogNormal Model)', fontsize=14, fontweight='bold')
ax.set_ylim(0, df['frp'].max() * 1.5)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

Sampling: [intercept, likelihood, sigma, slope]



```

# Fit LogNormal model - this takes a bit but worth it
with lognormal_model:
    # MCMC sampling - 2000 draws after 1000 tune iterations
    trace_lognormal = pm.sample(
        2000,
        tune=1000, # Warmup
        chains=4, # 4 chains for diagnostics
        return_inferencedata=True,
        idata_kwargs={"log_likelihood": True}, # Need this for L00 later
        random_seed=42
    )
    # Generate posterior predictions for model checking
    pm.sample_posterior_predictive(trace_lognormal, extend_inferencedata=True,
                                  random_seed=42)

```



```

Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [intercept, slope, sigma]
/Users/maradumitru/MuonSpace-Prototype/.venv/lib/python3.10/site-packages/rich/live.py:256: UserWarning: install
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')

Sampling 4 chains for 1_000 tune and 2_000 draw iterations (4_000 + 8_000 draws total) took 3 seconds.
Sampling: [likelihood]
/Users/maradumitru/MuonSpace-Prototype/.venv/lib/python3.10/site-packages/rich/live.py:256: UserWarning: install
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')

```

```

# Plot posterior and posterior-predictive distributions
fig, ax = plt.subplots(figsize=(12, 7))

# Plot data
ax.scatter(df['brightness'], df['frp'], alpha=0.6, s=100, c='#9370DB',
           label='Data', zorder=3, edgecolors='black', linewidth=0.5)

# Create x-range for predictions
x_range = np.linspace(df['brightness'].min(), df['brightness'].max(), 100)
x_scaled = (x_range - brightness_mean) / brightness_std

# Compute posterior distribution of  $\mu$  (the regression line)
# Using our helper function to avoid repetition
mu_posterior = compute_mu_posterior(trace_lognormal, x_scaled)

# Plot OUTER band: Posterior Predictive 95% HDI (prediction uncertainty)
az.plot_hdi(df['brightness'], trace_lognormal.posterior_predictive['likelihood'],
            hdi_prob=0.95, smooth=False, color='red',
            fill_kwargs={'alpha': 0.2, 'label': 'Posterior-Predictive 95% HDI'})

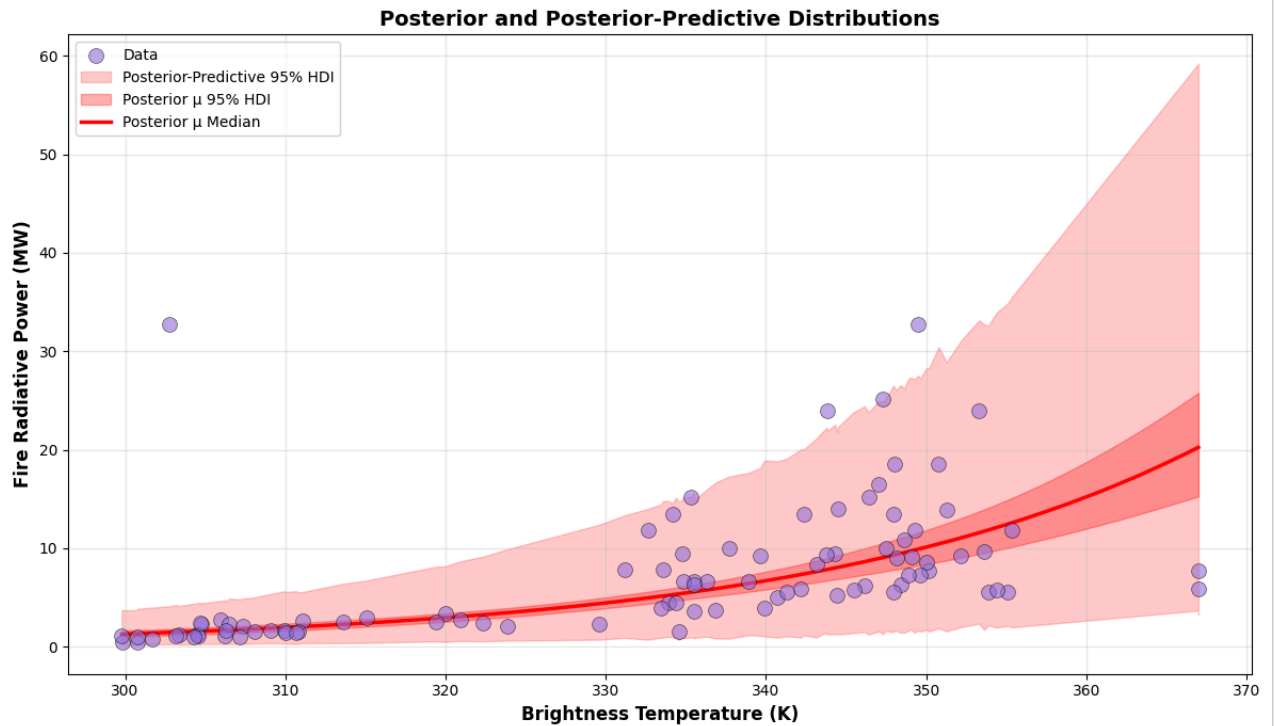
# Plot INNER band: Posterior  $\mu$  95% HDI (parameter uncertainty)
az.plot_hdi(x_range, mu_posterior, hdi_prob=0.95,
            color='red', fill_kwargs={'alpha': 0.3, 'label': 'Posterior  $\mu$  95% HDI'})

# Plot posterior median line - central estimate
mu_median = np.median(mu_posterior, axis=0)
ax.plot(x_range, mu_median, color='red', linewidth=2.5, label='Posterior  $\mu$  Median', zorder=2)

ax.set_xlabel('Brightness Temperature (K)', fontsize=12, fontweight='bold')
ax.set_ylabel('Fire Radiative Power (MW)', fontsize=12, fontweight='bold')
ax.set_title('Posterior and Posterior-Predictive Distributions', fontsize=14, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

/Users/maradumitru/MuonSpace-Prototype/.venv/lib/python3.10/site-packages/arviz/plots/hdiplot.py:166: FutureWarning
 hdi_data = hdi(y, hdi_prob=hdi_prob, circular=circular, multimodal=False, **hdi_kwargs)



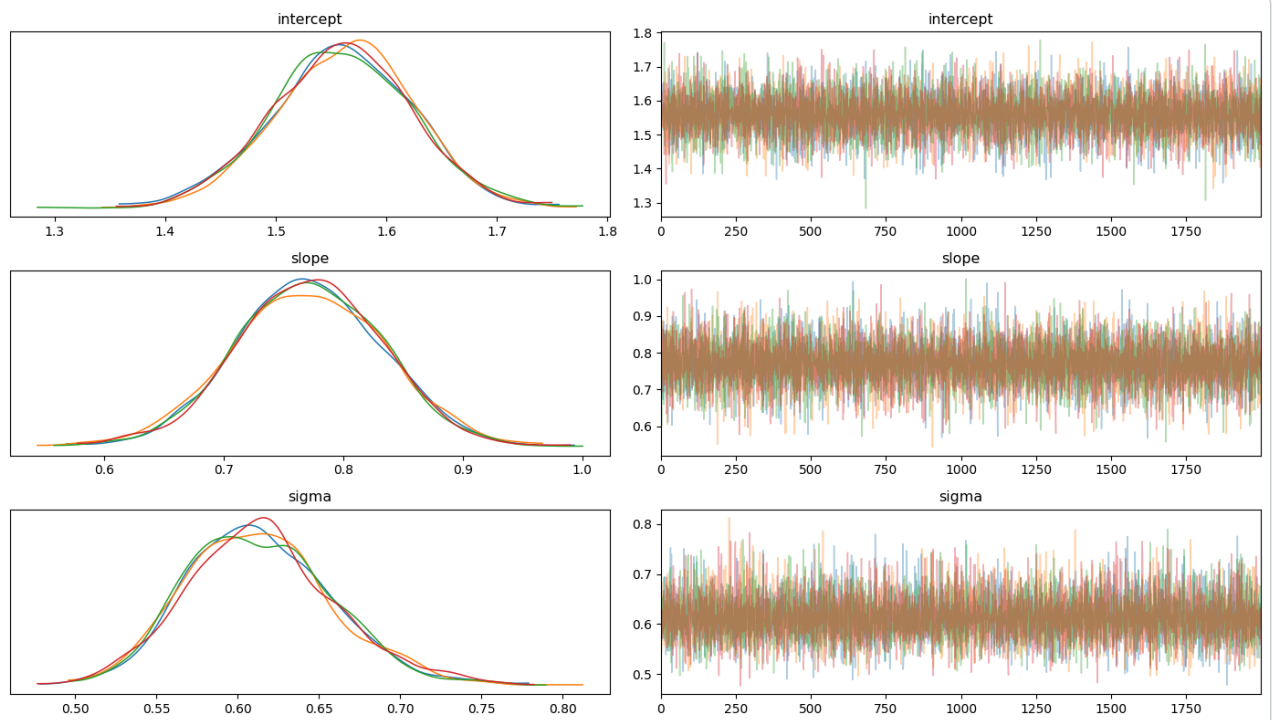
```
# Posterior predictive check - see how many obs fall in the 95% HDI
post_pred = trace_lognormal.posterior_predictive['likelihood'].values
post_pred_flat = post_pred.reshape(-1, len(df)) # Flatten chains

# Check coverage for each data point
coverage_count = 0
for i in range(len(df)):
    obs_value = df['frp'].values[i]
    pred_samples = post_pred_flat[:, i] # All posterior samples for this observation
    hdi = np.percentile(pred_samples, [2.5, 97.5]) # 95% HDI bounds
    if hdi[0] <= obs_value <= hdi[1]:
        coverage_count += 1 # Count if observation is within HDI

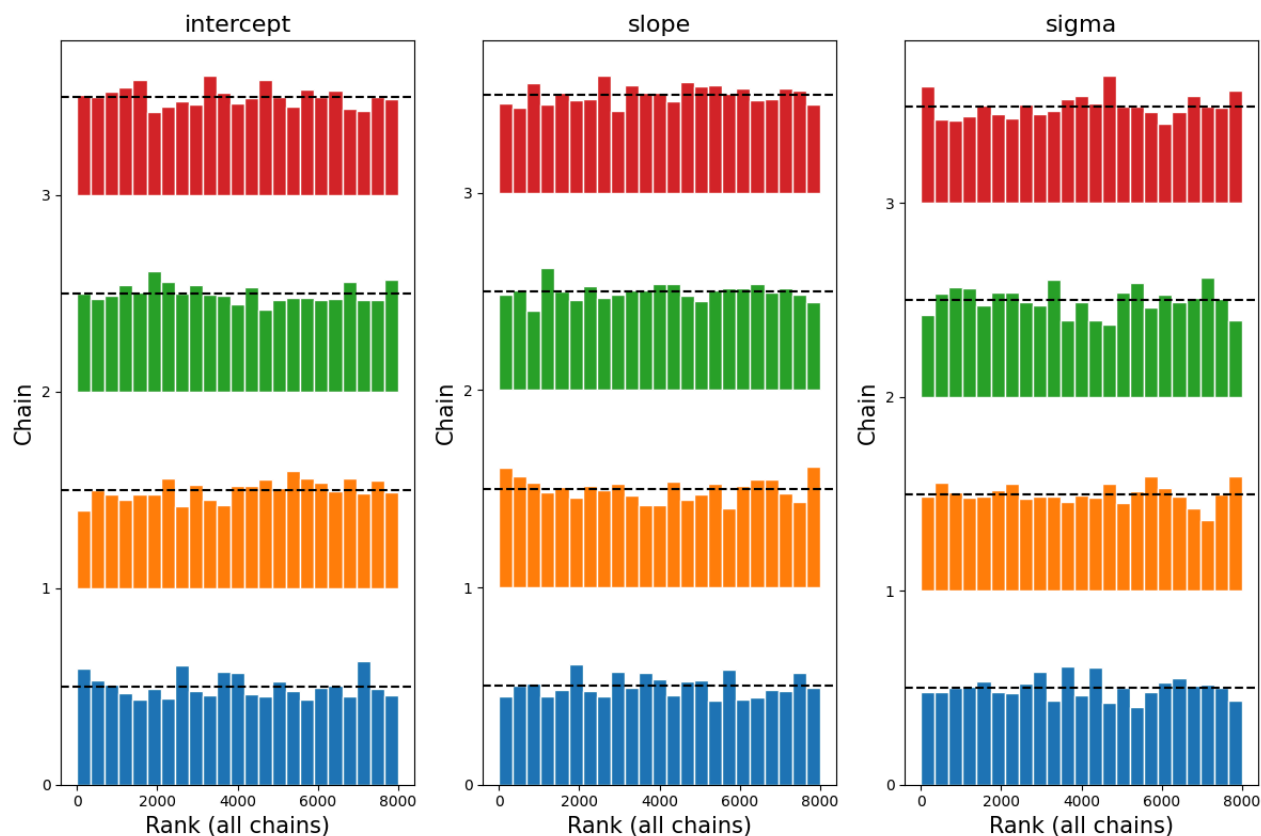
coverage_percentage = (coverage_count / len(df)) * 100
print(f"Observed data points within 95% HDI: {coverage_count} / {len(df)}")
print(f"Coverage percentage: {coverage_percentage:.1f}%")
```

```
Observed data points within 95% HDI: 92 / 94
Coverage percentage: 97.9%
```

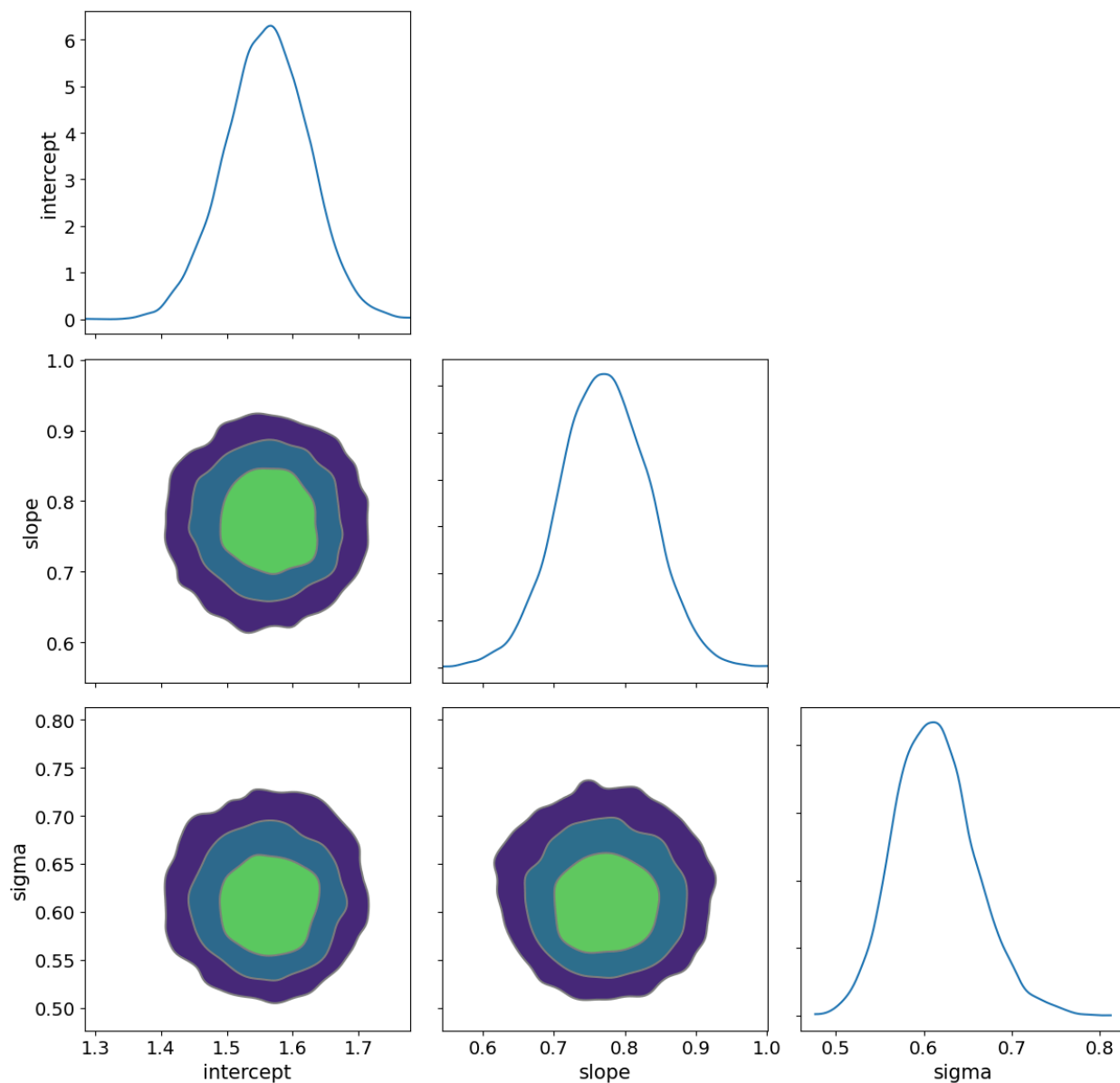
```
# Trace plots - Convergence Diagnostics
# Good way to check if chains mixed well and converged
az.plot_trace(trace_lognormal, var_names=['intercept', 'slope', 'sigma'],
              compact=False, figsize=(14, 8))
plt.tight_layout()
plt.show()
```



```
# Rank plots - Convergence Diagnostics
# Rank plots check if chains are mixing properly (should be uniform)
az.plot_rank(trace_lognormal,
             var_names=['intercept', 'slope', 'sigma'],
             kind='bars',
             figsize=(12, 8))
plt.tight_layout()
plt.show()
```



```
# Pair plot - Parameter Correlations
# Shows how parameters relate to each other - useful for checking identifiability
az.plot_pair(trace_lognormal,
             var_names=['intercept', 'slope', 'sigma'],
             kind='kde',
             marginals=True,
             figsize=(12, 12),
             divergences=True) # Highlight any divergences
plt.suptitle('LogNormal Model: Parameter Correlations',
             fontsize=16, fontweight='bold', y=0.995)
plt.tight_layout()
plt.show()
```

LogNormal Model: Parameter Correlations

```
# LogNormal Model Parameter Diagnostics
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

ax = axes[0]
az.plot_posterior(trace_lognormal, var_names=['intercept'], ax=ax, color='blue',
                  hdi_prob=0.95)
ax.set_title('Intercept (Log Scale)', fontweight='bold', fontsize=12)

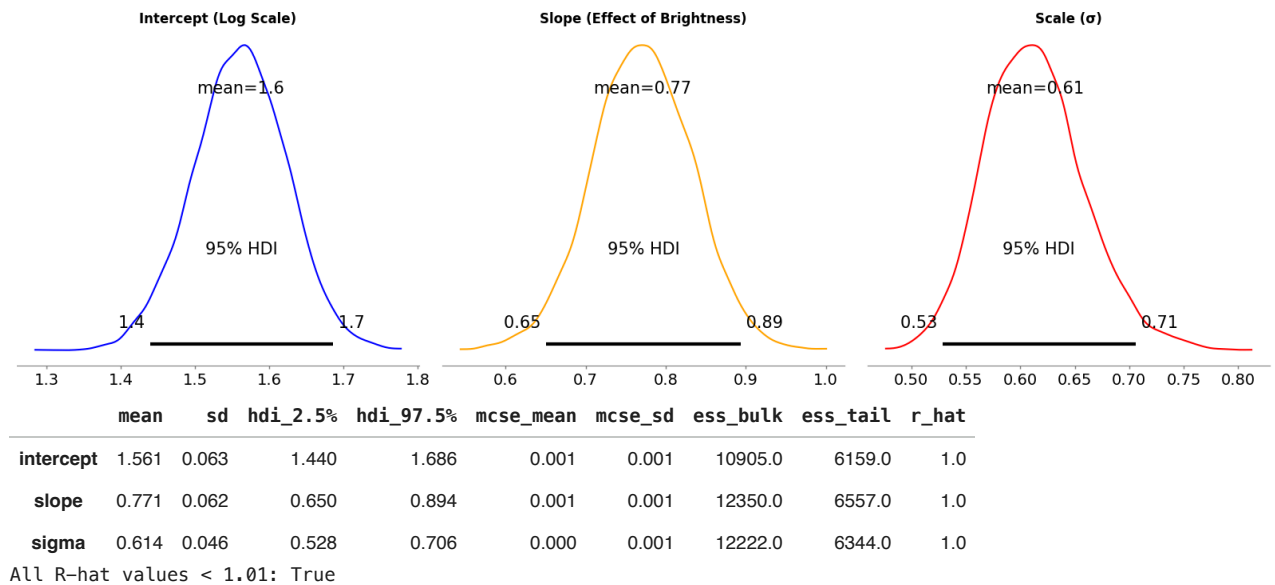
ax = axes[1]
az.plot_posterior(trace_lognormal, var_names=['slope'], ax=ax, color='orange',
                  hdi_prob=0.95)
ax.set_title('Slope (Effect of Brightness)', fontweight='bold', fontsize=12)

ax = axes[2]
az.plot_posterior(trace_lognormal, var_names=['sigma'], ax=ax, color='red',
                  hdi_prob=0.95)
ax.set_title('Scale ( $\sigma$ )', fontweight='bold', fontsize=12)
```

```
plt.tight_layout()
plt.show()

# Quick summary to check convergence looks good
summary = az.summary(trace_lognormal, var_names=['intercept', 'slope', 'sigma'],
                    hdi_prob=0.95)

display(summary)
print(f"All R-hat values < 1.01: {(summary['r_hat'] < 1.01).all()}")
```



```
# Outlier sensitivity test: refit LogNormal model without outlier
# Trying to see how much that one weird point affects things
outlier_idx = df[(df['brightness'] < 305) & (df['frp'] > 30)].index

if len(outlier_idx) > 0:
    print(f"Outlier identified at index {outlier_idx.values}: "
          f"Brightness={df.loc[outlier_idx, 'brightness'].values[0]:.1f}K, "
          f"FRP={df.loc[outlier_idx, 'frp'].values[0]:.2f}MW")

df_no_outlier = df.drop(outlier_idx)
# Re-standardize without the outlier (using same mean/std as original)
brightness_scaled_no_outlier = (df_no_outlier['brightness'] - brightness_mean) / brightness_std

print(f"Dataset sizes: Original: {len(df)} fires, Without outlier: {len(df_no_outlier)} fires")

# Refit model without the outlier to compare
with pm.Model() as lognormal_model_no_outlier_test:
    intercept = pm.Normal("intercept", mu=2.5, sigma=1)
    slope = pm.Normal("slope", mu=0, sigma=1)
    sigma = pm.HalfNormal("sigma", sigma=1)
    mu = pm.Deterministic("mu", intercept + slope * brightness_scaled_no_outlier)
    likelihood = pm.Lognormal("likelihood", mu=mu, sigma=sigma,
                              observed=df_no_outlier['frp'].values)

    trace_lognormal_no_outlier_test = pm.sample(
        2000, tune=1000, chains=4,
        return_inferencedata=True,
        random_seed=42
    )

# Extract posterior means for comparison
params_with = {
    'intercept': trace_lognormal.posterior['intercept'].mean().values,
    'slope': trace_lognormal.posterior['slope'].mean().values,
    'sigma': trace_lognormal.posterior['sigma'].mean().values
}
```

```

params_without = {
    'intercept': trace_lognormal_no_outlier_test.posterior['intercept'].mean().values,
    'slope': trace_lognormal_no_outlier_test.posterior['slope'].mean().values,
    'sigma': trace_lognormal_no_outlier_test.posterior['sigma'].mean().values
}

# Print comparison table
print(f"\nParameter comparison:")
print(f"{'Parameter':<15} {'With Outlier':<15} {'Without Outlier':<15} {'% Change':<15}")
print("-" * 65)
for param in ['intercept', 'slope', 'sigma']:
    change = params_without[param] - params_with[param]
    pct_change = (change / params_with[param]) * 100 # Percent change
    print(f"{'param':<15} {'params_with[param]:<15.4f} {'params_without[param]:<15.4f} "
          f"{'pct_change:+.2f}%"}")
else:
    print("No extreme outlier found matching criteria.")

```

Initializing NUTS using jitter+adapt_diag...

Outlier identified at index [3680426]: Brightness=302.8K, FRP=32.72MW

Dataset sizes: Original: 94 fires, Without outlier: 93 fires

Multiprocess sampling (4 chains in 4 jobs)

NUTS: [intercept, slope, sigma]

/Users/maradumitru/MuonSpace-Prototype/.venv/lib/python3.10/site-packages/rich/live.py:256: UserWarning: install

"ipywidgets" for Jupyter support

warnings.warn('install "ipywidgets" for Jupyter support')

Sampling 4 chains for 1_000 tune and 2_000 draw iterations (4_000 + 8_000 draws total) took 3 seconds.

Parameter comparison:

Parameter	With Outlier	Without Outlier	% Change
intercept	1.5607	1.5263	-2.20%
slope	0.7709	0.8242	+6.91%
sigma	0.6135	0.5173	-15.69%

Model 2: Robust (Student-t) Regression

This should handle outliers better than lognormal

from tabulate import tabulate

with pm.Model() as robust_model:

intercept = pm.Normal("intercept", mu=2.5, sigma=1)

slope = pm.Normal("slope", mu=0, sigma=1)

sigma = pm.HalfNormal("sigma", sigma=1)

nu controls how heavy the tails are - lower = more robust

nu = pm.Exponential("nu", 1/10)

mu = pm.Deterministic("mu", intercept + slope * brightness_scaled)

Using StudentT on log(FRP) to handle outliers better

```

likelihood = pm.StudentT(
    "likelihood",
    nu=nu,
    mu=mu,
    sigma=sigma,
    observed=np.log(df['frp'].values)
)

```

```

trace_robust = pm.sample(
    2000,
    tune=1000,
    chains=4,
    return_inferencedata=True,
    idata_kwargs={"log_likelihood": True},
    random_seed=42
)
pm.sample_posterior_predictive(trace_robust, extend_inferencedata=True, random_seed=42)

```

Check what nu learned - lower means heavier tails

nu_mean = trace_robust.posterior['nu'].mean().values

print(f"Learned degrees of freedom: v = {nu_mean:.2f}")

```

Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [intercept, slope, sigma, nu]
/Users/maradumitru/MuonSpace-Prototype/.venv/lib/python3.10/site-packages/rich/live.py:256: UserWarning: install
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')

Sampling 4 chains for 1_000 tune and 2_000 draw iterations (4_000 + 8_000 draws total) took 2 seconds.
Sampling: [likelihood]
/Users/maradumitru/MuonSpace-Prototype/.venv/lib/python3.10/site-packages/rich/live.py:256: UserWarning: install
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')

```

```

# Diagnostics for the robust model - check convergence and quality
print("=== Robust Model Diagnostics ===")

# Summary statistics table (like the one shown for the previous model)
summary_stats = az.summary(trace_robust, var_names=['intercept', 'slope', 'sigma', 'nu'])
print(summary_stats)

# Check if all R-hat values are good
all_rhat_good = (summary_stats['r_hat'] < 1.01).all()
print(f"\nAll R-hat values < 1.01: {all_rhat_good}")

# 3. Rank plots - should look uniform if chains are mixing well
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for i, var in enumerate(['intercept', 'slope', 'sigma', 'nu']):
    az.plot_rank(trace_robust, var_names=[var], ax=axes[i])
    axes[i].set_title(f'Rank plot: {var}')

plt.tight_layout()
plt.suptitle('Rank Plots for Robust Model\n(Should look uniform for good mixing)',
            fontsize=14, y=1.02)
plt.show()

```

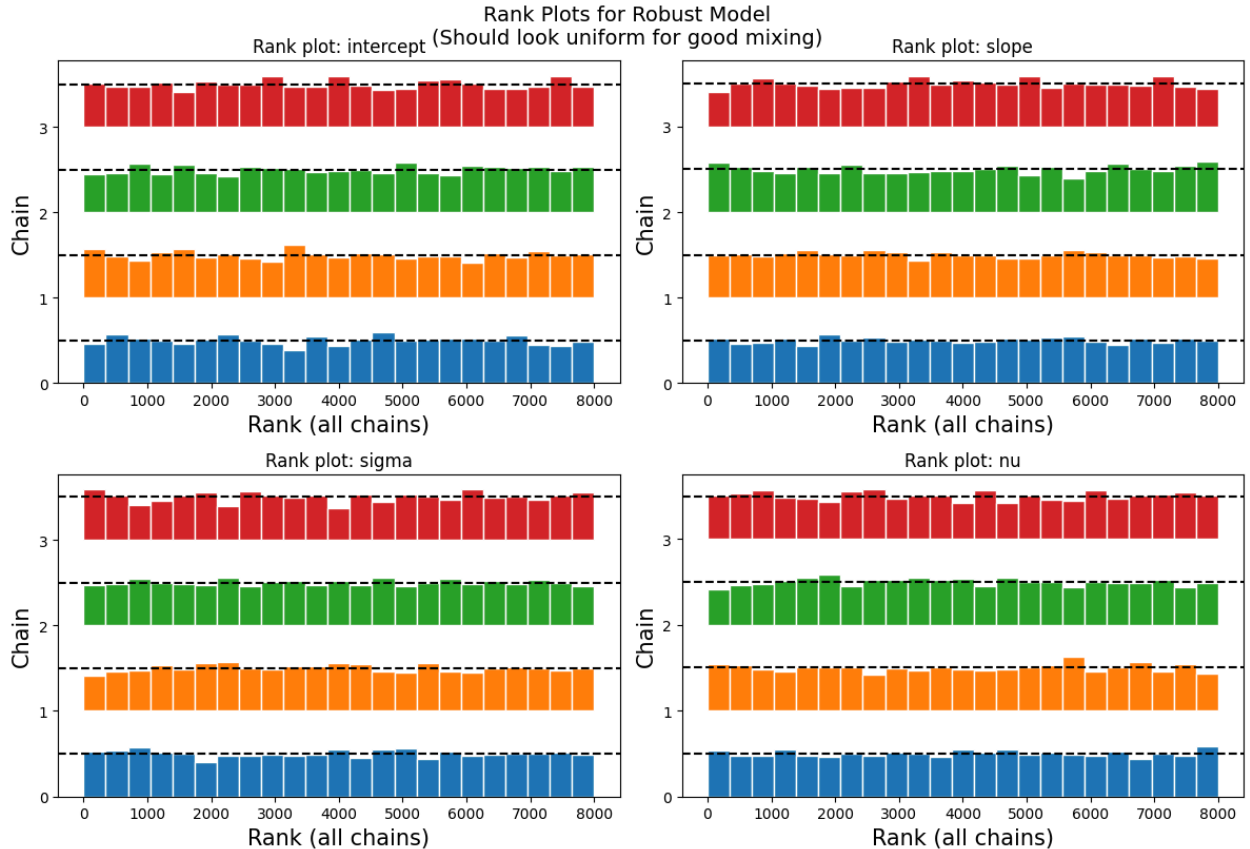


```
=== Robust Model Diagnostics ===
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
intercept	1.522	0.054	1.420	1.623	0.001	0.001	7404.0	
slope	0.820	0.055	0.710	0.917	0.001	0.001	7523.0	
sigma	0.458	0.055	0.356	0.558	0.001	0.001	5217.0	
nu	6.104	3.427	1.771	11.542	0.050	0.102	5421.0	

	ess_tail	r_hat
intercept	5841.0	1.0
slope	6406.0	1.0
sigma	5109.0	1.0
nu	5059.0	1.0

All R-hat values < 1.01: True



```
# Model comparison using LOO-CV - this tells us which model is actually better
comparison = az.compare(
    {'LogNormal': trace_lognormal, 'Robust (Student-t)': trace_robust},
    ic='loo', scale='log'
)

# Clean up the comparison table for display
comparison_display = comparison.reset_index().rename(columns={
    'index': 'Model',
    'elpd_loo': 'elpd_loo',
    'se': 'SE',
    'p_loo': 'p_loo',
    'elpd_diff': 'Δelpd',
    'weight': 'Weight'
})

# Add rank column and round values for readability
comparison_display.insert(1, 'Rank', range(len(comparison_display)))
comparison_display[['elpd_loo', 'SE', 'p_loo', 'Δelpd', 'Weight']] = comparison_display[
    ['elpd_loo', 'SE', 'p_loo', 'Δelpd', 'Weight']
].round(2)

# Pretty print the comparison table
print(tabulate(comparison_display, headers='keys', tablefmt='github', showindex=False))
```

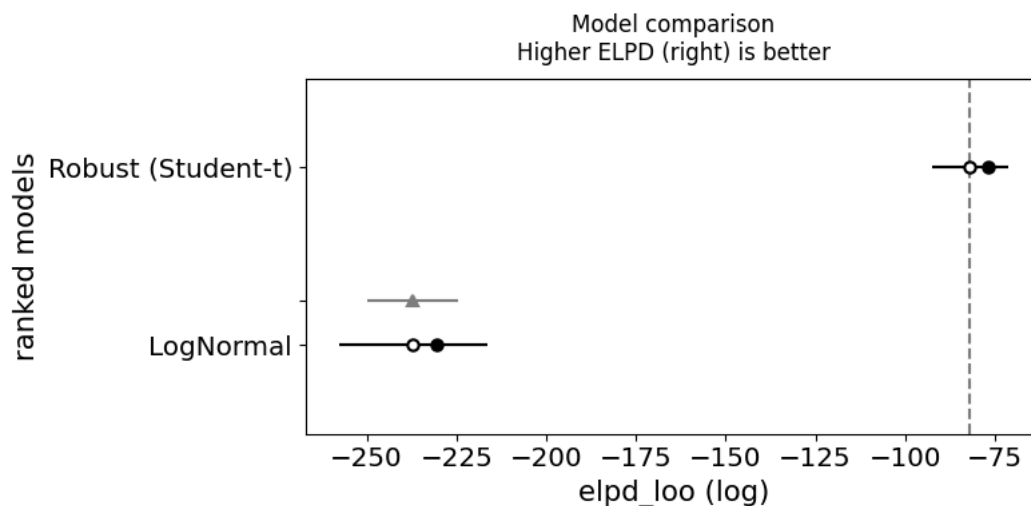
```
# Extract best model info
best_model = comparison_display.iloc[0]['Model']
second_model = comparison_display.iloc[1]['Model']
elpd_diff = abs(comparison_display.iloc[1]['Δelpd']) # Difference in ELPD
se_diff = comparison.iloc[1]['dse'] # Standard error of the difference

print(f"\nELPD Difference: {elpd_diff:.2f} ± {se_diff:.2f} (favors {best_model})")
```

Model	Rank	rank	elpd_loo	p_loo	Δelpd	Weight	SE	dse	warning
Robust (Student-t)	0	0	-81.95	4.91	0	1	10.62	0	False
LogNormal	1	1	-237.08	6.69	155.13	0	20.63	12.6397	True

ELPD Difference: 155.13 ± 12.64 (favors Robust (Student-t))
 /Users/maradumitru/MuonSpace-Prototype/.venv/lib/python3.10/site-packages/arviz/stats/stats.py:797: UserWarning:
 warnings.warn()

```
# Visual model comparison
# Nice plot showing ELPD comparison with uncertainty
az.plot_compare(
    comparison,
    figsize=(8, 4),
    insample_dev=True,
    plot_ic_diff=True # Show difference from best model
)
plt.title("Model comparison\nHigher ELPD (right) is better", fontsize=12, pad=10)
plt.tight_layout()
plt.show()
```



```
# Outlier influence analysis: refit both models without outlier
outlier_idx = df[(df['brightness'] < 305) & (df['frp'] > 30)].index

if len(outlier_idx) > 0:
    print(f"Outlier found: Brightness={df.loc[outlier_idx, 'brightness'].values[0]:.1f}K, "
          f"FRP={df.loc[outlier_idx, 'frp'].values[0]:.2f}MW")

    df_no_outlier = df.drop(outlier_idx)
    # Use same standardization as before for fair comparison
    brightness_scaled_no_outlier = (df_no_outlier['brightness'] - brightness_mean) / brightness_std

    print(f"Original dataset: {len(df)} fires, Without outlier: {len(df_no_outlier)} fires")

    # Refit LogNormal model without outlier
    # Same priors and setup as before
    with pm.Model() as lognormal_model_no_outlier:
        intercept = pm.Normal("intercept", mu=2.5, sigma=1)
        slope = pm.Normal("slope", mu=0, sigma=1)
        sigma = pm.HalfNormal("sigma", sigma=1)
        mu = pm.Deterministic("mu", intercept + slope * brightness_scaled_no_outlier)
        likelihood = pm.Lognormal("likelihood", mu=mu, sigma=sigma,
                                   observed=df_no_outlier['frp'].values)

    trace_lognormal_no_outlier = pm.sample(
        2000, tune=1000, chains=4,
        return_inferencedata=True,
        idata_kwargs={"log_likelihood": True},
        random_seed=42
```

```

    )

# Refit Robust model without outlier - same setup
with pm.Model() as robust_model_no_outlier:
    intercept = pm.Normal("intercept", mu=2.5, sigma=1)
    slope = pm.Normal("slope", mu=0, sigma=1)
    sigma = pm.HalfNormal("sigma", sigma=1)
    nu = pm.Exponential("nu", 1/10)
    mu = pm.Deterministic("mu", intercept + slope * brightness_scaled_no_outlier)
    likelihood = pm.StudentT("likelihood", nu=nu, mu=mu, sigma=sigma,
                             observed=np.log(df_no_outlier['frp'].values))

    trace_robust_no_outlier = pm.sample(
        2000, tune=1000, chains=4,
        return_inferencedata=True,
        idata_kwargs={"log_likelihood": True},
        random_seed=42
    )

# Helper function to extract mean params - makes the comparison cleaner
def get_params(trace):
    return {
        'intercept': trace.posterior['intercept'].mean().values,
        'slope': trace.posterior['slope'].mean().values,
        'sigma': trace.posterior['sigma'].mean().values
    }

# Get parameters for comparison
params_ln_with = get_params(trace_lognormal)
params_ln_without = get_params(trace_lognormal_no_outlier)
params_rb_with = get_params(trace_robust)
params_rb_without = get_params(trace_robust_no_outlier)

# Print detailed comparison tables
print(f"\nLOGNORMAL MODEL:")
print(f'{"Parameter":<15} {"With Outlier":<15} {"Without Outlier":<15} {"Change":<15}')
print("-" * 60)
for param in ['intercept', 'slope', 'sigma']:
    change = params_ln_without[param] - params_ln_with[param]
    pct_change = (change / params_ln_with[param]) * 100
    print(f'{param:<15} {params_ln_with[param]:<15.4f} {params_ln_without[param]:<15.4f} '
          f'{{change:+.4f}} ({{pct_change:+.1f}}%)')

print(f"\nROBUST (STUDENT-T) MODEL:")
print(f'{"Parameter":<15} {"With Outlier":<15} {"Without Outlier":<15} {"Change":<15}')
print("-" * 60)
for param in ['intercept', 'slope', 'sigma']:
    change = params_rb_without[param] - params_rb_with[param]
    pct_change = (change / params_rb_with[param]) * 100
    print(f'{param:<15} {params_rb_with[param]:<15.4f} {params_rb_without[param]:<15.4f} '
          f'{{change:+.4f}} ({{pct_change:+.1f}}%)')

# Also compare nu (degrees of freedom) for robust model
nu_with = trace_robust.posterior['nu'].mean().values
nu_without = trace_robust_no_outlier.posterior['nu'].mean().values
print(f'{"nu (df)":<15} {nu_with:<15.4f} {nu_without:<15.4f} '
      f'{{nu_without - nu_with:+.4f}} ({{(nu_without - nu_with) / nu_with * 100:+.1f}}%)')
else:
    print("No extreme outlier found matching criteria.")

Outlier found: Brightness=302.8K, FRP=32.72MW
Original dataset: 94 fires, Without outlier: 93 fires
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [intercept, slope, sigma]
/Users/maradumitru/MuonSpace-Prototype/.venv/lib/python3.10/site-packages/rich/live.py:256: UserWarning: install
"ipywidgets" for Jupyter support
warnings.warn('install "ipywidgets" for Jupyter support')

# Visual comparison: LogNormal vs Robust with/without outlier
if len(outlier_idx) > 0:
    fig, axes = plt.subplots(1, 2, figsize=(16, 7))

    # LogNormal comparison
    ax = axes[0]
    ax.scatter(df['brightness'], df['frp'], alpha=0.6, s=100, c='#9370DB',
              label='All Data', zorder=3, edgecolors='black', linewidth=0.5)

    # Highlight the outlier with big red X
    ax.scatter(df.loc[outlier_idx, 'brightness'], df.loc[outlier_idx, 'frp'],
              alpha=1.0, s=200, c='red', marker='X',
              label='Outlier (excluded)', zorder=4, edgecolors='black', linewidth=1.5)

```

```

x_range = np.linspace(df['brightness'].min(), df['brightness'].max(), 100)
x_scaled = (x_range - brightness_mean) / brightness_std

# Compute posterior means for both fits
mu_with = compute_mu_posterior(trace_lognormal, x_scaled, max_samples=1000)
mu_without = compute_mu_posterior(trace_lognormal_no_outlier, x_scaled, max_samples=1000)

# Plot median predictions for comparison
ax.plot(x_range, np.median(mu_with, axis=0), color='red', linewidth=3,
        label='With Outlier', linestyle='--', zorder=2)
ax.plot(x_range, np.median(mu_without, axis=0), color='blue', linewidth=3,
        label='Without Outlier', zorder=2)

ax.set_xlabel('Brightness Temperature (K)', fontsize=12, fontweight='bold')
ax.set_ylabel('Fire Radiative Power (MW)', fontsize=12, fontweight='bold')
ax.set_title('LogNormal: Outlier Influence', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)

# Robust comparison
ax = axes[1]
ax.scatter(df['brightness'], df['frp'], alpha=0.6, s=100, c='#9370DB',
          label='All Data', zorder=3, edgecolors='black', linewidth=0.5)
ax.scatter(df.loc[outlier_idx, 'brightness'], df.loc[outlier_idx, 'frp'],
          alpha=1.0, s=200, c='red', marker='X',
          label='Outlier (excluded)', zorder=4, edgecolors='black', linewidth=1.5)

mu_rb_with = compute_mu_posterior(trace_robust, x_scaled, max_samples=1000)
mu_rb_without = compute_mu_posterior(trace_robust_no_outlier, x_scaled, max_samples=1000)

ax.plot(x_range, np.median(mu_rb_with, axis=0), color='darkgreen', linewidth=3,
        label='With Outlier', linestyle='--', zorder=2)
ax.plot(x_range, np.median(mu_rb_without, axis=0), color='blue', linewidth=3,
        label='Without Outlier', zorder=2)

ax.set_xlabel('Brightness Temperature (K)', fontsize=12, fontweight='bold')
ax.set_ylabel('Fire Radiative Power (MW)', fontsize=12, fontweight='bold')
ax.set_title('Robust (Student-t): Outlier Influence', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```